



School of Computer, Information and Mathematical Sciences

DEPARTMENT OF INFORMATION TECHNOLOGY

ITD 2102 Programming in Python

III Sem B.Tech. IT – ‘B’ Sec

A MINI PROJECT REPORT

on

Cloud-Based File Encryption and Sharing Platform

Submitted by

Team	RRN	Name	Report (10)	Program & Demo (30)	Viva (10)	Total (50)
T2	240081601057	Ashwin A				
	240081601059	Faheem S				
	240081601060	Harshavardhan K				

Date: 07.11.2025

Course Teacher

Dr. N. Rajendran
Associate Professor/ IT

Cloud-Based File Encryption and Sharing Platform

1. Introduction:

This project is a Cloud-Based File Encryption and Sharing Platform built using Python. It allows users to securely upload, encrypt, and share files with others. Each uploaded file is automatically encrypted using a secret key, and users can share this key with others to allow secure file access. The system also includes user login, registration, password protection, and an admin panel for managing users and data. The main aim is to provide a safe, easy-to-use environment for file storage and sharing using strong encryption techniques.

2. Objectives:

- To design a simple secure cloud storage system using Python.
- To encrypt files before storing them, ensuring data privacy.
- To allow users to upload, download, and share files securely.
- To provide an admin panel for managing users, logs, and files.
- To implement password protection, backup, and file integrity checking.
- To demonstrate how Python can be used for real-world data security applications.

3. System Requirements

Hardware Requirements

- **Processor:** Intel i3 or higher
- **RAM:** Minimum 4 GB
- **Hard Disk:** 100MB free space

Software Requirements

- Python 3.8 or above
- Any Python-supported IDE
- **Libraries Used:**
 - cryptography – for encryption and decryption
 - hashlib, json, os, shutil, datetime, getpass
- **Text Editor / IDE:** VS Code, PyCharm, or any Python IDE

4. Proposed System

The proposed system provides a secure platform for users to store and share files using encryption. When a user uploads a file, it is automatically encrypted and stored securely in the cloud folder. Only users with the correct encryption key can decrypt and download the file. The system also maintains user logs, metadata, and password protection to ensure security and accountability. An admin user can view logs, reset passwords.

5. Functional Modules

- User Registration & Login: Secure authentication using SHA-256 password hashing.
- File Encryption & Upload: Encrypt files using Fernet symmetric encryption before saving.
- File Download & Decryption: Decrypt files securely using the correct key.
- File Sharing: Share encryption keys with other users for controlled access.
- Admin Panel: Manage users, view logs, perform maintenance, and monitor system activity.
- Backup & Recovery: Generate backups for user keys and system metadata.

6. Code:

main.py -

```
import os
import sys
import hashlib
from cryptography.fernet import Fernet, InvalidToken
from datetime import datetime
import shutil
import getpass
import warnings
import re
import json
import time

class CEncryptLogic:
    def __init__(self):
        self.BASE_DIR = os.path.abspath(os.path.dirname(__file__))
        self.CSTORAGE_DIR = os.path.join(self.BASE_DIR, 'cstorage')
        self.ENCRYPTED_DIR = os.path.join(self.CSTORAGE_DIR, 'encrypted')
        self.KEYS_DIR = os.path.join(self.CSTORAGE_DIR, 'keys')
        self.DECRYPTED_DIR = os.path.join(self.CSTORAGE_DIR, 'decrypted')
        self.BACKUP_DIR = os.path.join(self.CSTORAGE_DIR, 'backups')
        self.USERS_FILE = os.path.join(self.CSTORAGE_DIR, 'users.txt')
        self.LOGS_FILE = os.path.join(self.CSTORAGE_DIR, 'logs.txt')
        self.METADATA_FILE = os.path.join(self.CSTORAGE_DIR, 'metadata.json')
        self.MAINTENANCE_FILE = os.path.join(self.CSTORAGE_DIR, 'maintenance.flag')
        self.ADMIN_USER = "Admin"
        self.ADMIN_PASS = "admin"
        self.ensure_dirs()
```

```

        self.create_admin()
def ensure_dirs(self):
    try:
        for directory in [self.CSTORAGE_DIR, self.ENCRYPTED_DIR, self.KEYS_DIR,
                           self.DECRYPTED_DIR, self.BACKUP_DIR]:
            os.makedirs(directory, exist_ok=True)
        for file_path in [self.USERS_FILE, self.LOGS_FILE]:
            if not os.path.exists(file_path):
                open(file_path, 'a').close()
        if not os.path.exists(self.METADATA_FILE):
            with open(self.METADATA_FILE, 'w') as f:
                json.dump({}, f)
    except Exception as e:
        print(f"Error creating directories: {e}")
        sys.exit(1)
def create_admin(self):
    if not self._check_user_exists(self.ADMIN_USER):
        hashed = self._hash_password(self.ADMIN_PASS)
        with open(self.USERS_FILE, 'a') as f:
            f.write(f"{self.ADMIN_USER}:{hashed}:ADMIN:false\n")
def _hash_password(self, password):
    return hashlib.sha256(password.encode()).hexdigest()
def _log_action(self, username, action, details):
    try:
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        log_entry = f"[{timestamp}] - USER: {username} - ACTION: {action} - DETAILS:
{details}\n"
        with open(self.LOGS_FILE, 'a') as f:
            f.write(log_entry)
    except Exception:
        pass
def check_password_strength(self, password):
    errors = []
    if len(password) < 8:
        errors.append("Password must be at least 8 characters long")
    if not re.search(r'[A-Z]', password):
        errors.append("Password must contain at least one uppercase letter")
    if not re.search(r'[0-9]', password):
        errors.append("Password must contain at least one number")
    return (True, []) if not errors else (False, errors)
def register_user(self, username, password):
    if not username or not password:
        return "Username and password cannot be empty."
    if ' ' in username or ':' in username:
        return "Username cannot contain spaces or colons."
    is_strong, issues = self.check_password_strength(password)
    if not is_strong:
        return "Password requirements not met:\n" + "\n".join(f" * {issue}" for issue in
issues)
    if self._check_user_exists(username):
        return "Username already exists."
    hashed_password = self._hash_password(password)
    with open(self.USERS_FILE, 'a') as f:
        f.write(f"{username}:{hashed_password}:USER:false\n")
    self._log_action(username, "REGISTER", "New user created")
    return "Success"
def login_user(self, username, password):
    if not os.path.exists(self.USERS_FILE):
        return False, None, "No users registered", False
    user_found = False
    hashed_password = self._hash_password(password)
    with open(self.USERS_FILE, 'r') as f:
        for line in f:
            parts = line.strip().split(':')
            if len(parts) >= 2:
                stored_user, stored_hash = parts[0], parts[1]
                if stored_user == username:

```

```

        user_found = True
        role = parts[2] if len(parts) > 2 else "USER"
        reset_required = parts[3] == "true" if len(parts) > 3 else False
        if stored_hash == hashed_password:
            self._log_action(username, "LOGIN", f"User logged in as {role}")
            return True, role, None, reset_required
        else:
            self._log_action(username, "LOGIN_FAIL", "Incorrect password")
            return False, None, "Incorrect password", False

    if not user_found:
        self._log_action(username, "LOGIN_FAIL", "User not found")
        return False, None, "User not found", False
    return False, None, "Login failed", False

def get_all_users(self):
    try:
        users = []
        with open(self.USERS_FILE, 'r') as f:
            for line in f:
                parts = line.strip().split(':')
                if len(parts) >= 2:
                    username = parts[0]
                    role = parts[2] if len(parts) > 2 else "USER"
                    users.append(f"{username} ({role})")
        return users
    except Exception:
        return []

def get_all_files(self):
    try:
        return os.listdir(self.ENCRYPTED_DIR)
    except Exception:
        return []

def upload_file(self, username, file_path):
    if self.is_maintenance_mode():
        return "Error: System is in maintenance mode. Uploads disabled."
    if not file_path or not os.path.exists(file_path):
        return "Error: File not found."
    try:
        file_size = os.path.getsize(file_path)
        key = Fernet.generate_key()
        cipher = Fernet(key)
        with open(file_path, 'rb') as file:
            file_data = file.read()
            encrypted_data = cipher.encrypt(file_data)
            file_hash = hashlib.sha256(encrypted_data).hexdigest()
            original_name = os.path.basename(file_path)
            final_name = self._get_unique_filename(username, original_name)
            encrypted_name = f"{username}_{final_name}.enc"
            key_name = f"{username}_{final_name}.key"
            with open(os.path.join(self.ENCRYPTED_DIR, encrypted_name), 'wb') as f:
                f.write(encrypted_data)
            with open(os.path.join(self.KEYS_DIR, key_name), 'wb') as f:
                f.write(key)
            self.save_file_metadata(username, final_name, file_size, file_hash)
            size_str = self._format_file_size(file_size)
            self._log_action(username, "UPLOAD", f"File: {final_name} ({size_str})")
            return f"Success! File uploaded as '{encrypted_name}'."
    except Exception as e:
        return f"Error: {e}"

def _get_unique_filename(self, username, filename):
    if '.' in filename:
        name, ext = filename.rsplit('.', 1)
        ext = '.' + ext
    else:
        name = filename
        ext = ''
    base_encrypted = f"{username}_{filename}.enc"
    if not os.path.exists(os.path.join(self.ENCRYPTED_DIR, base_encrypted)):

```

```

        return filename
    counter = 1
    while True:
        new_name = f"{name} ({counter}){ext}"
        test_encrypted = f"{username}_{new_name}.enc"
        if not os.path.exists(os.path.join(self.ENCRYPTED_DIR, test_encrypted)):
            return new_name
        counter += 1
    def get_user_files(self, username):
        try:
            all_files = os.listdir(self.ENCRYPTED_DIR)
            user_files = [f for f in all_files if f.startswith(f"{username}_") and
f.endswith(".enc")]
            shared_files = []
            user_keys = self.get_user_keys(username)
            for key_file in user_keys:
                if not key_file.startswith(f"{username}_"):
                    continue
                base_filename = key_file.replace(f"{username}_", "").replace(".key", "")
                for enc_file in all_files:
                    if not enc_file.endswith(".enc"):
                        continue
                    if enc_file.startswith(f"{username}_"):
                        continue
                    if '_' in enc_file:
                        enc_base = enc_file.split('_', 1)[1].replace(".enc", "")
                        if enc_base == base_filename and enc_file not in shared_files:
                            shared_files.append(enc_file)
            return user_files + shared_files
        except Exception:
            return []
    def download_file(self, username, encrypted_name, key_path):
        if self.is_maintenance_mode():
            return "Error: System is in maintenance mode. Downloads disabled."
        try:
            if not os.path.exists(key_path):
                return "Error: Key file not found."
            with open(key_path, 'rb') as f:
                key = f.read()
            cipher = Fernet(key)
            encrypted_path = os.path.join(self.ENCRYPTED_DIR, encrypted_name)
            if not os.path.exists(encrypted_path):
                return "Error: Encrypted file not found."
            with open(encrypted_path, 'rb') as f:
                encrypted_data = f.read()
            current_hash = hashlib.sha256(encrypted_data).hexdigest()
            original_name = encrypted_name.split('_', 1)[1].replace(".enc", "") if '_' in
encrypted_name else encrypted_name.replace(".enc", "")
            owner = encrypted_name.split('_')[0] if '_' in encrypted_name else username
            metadata = self.get_file_metadata(owner, original_name)
            if metadata and 'hash' in metadata:
                stored_hash = metadata['hash']
                if current_hash != stored_hash:
                    self._log_action(username, "INTEGRITY_FAIL", f"Hash mismatch for
{encrypted_name}")
                    return "Error: File integrity check failed. File may be corrupted."
            decrypted_data = cipher.decrypt(encrypted_data)
            decrypted_path = os.path.join(self.DECRYPTED_DIR, original_name)
            with open(decrypted_path, 'wb') as f:
                f.write(decrypted_data)
            self._log_action(username, "DOWNLOAD", f"File: {original_name}")
            return f"Success! File saved to 'decrypted' folder as '{original_name}'."
        except InvalidToken:
            self._log_action(username, "DOWNLOAD_FAIL", f"Invalid key for {encrypted_name}")
            return "Error: Wrong key. Decryption failed."
        except Exception as e:
            return f"Error: {e}"

```

```

def get_user_keys(self, username):
    try:
        files = os.listdir(self.KEYS_DIR)
        return [f for f in files if f.startswith(f"{username}_") and f.endswith(".key")]
    except Exception:
        return []

def _check_user_exists(self, username):
    if not os.path.exists(self.USERS_FILE):
        return False
    try:
        with open(self.USERS_FILE, 'r') as f:
            return any(line.strip().split(':')[0] == username for line in f)
    except Exception:
        return False

def share_key(self, username, key_name, target_user):
    if not target_user:
        return "Please enter a username."
    if target_user == username:
        return "Error: Cannot share with yourself."
    if not self._check_user_exists(target_user):
        return f"Error: User '{target_user}' does not exist."
    try:
        original_key = os.path.join(self.KEYS_DIR, key_name)
        if not os.path.exists(original_key):
            return "Error: Key file not found."
        file_part = key_name.replace(f"{username}_", "")
        new_key_path = os.path.join(self.KEYS_DIR, f"{target_user}_{file_part}")
        shutil.copy(original_key, new_key_path)
        self._log_action(username, "SHARE", f"Shared with {target_user}")
        return f"Success! Key shared with {target_user}."
    except Exception as e:
        return f"Error: {e}"

def delete_file(self, username, encrypted_name):
    if not encrypted_name.startswith(f"{username}_"):
        return "Error: Not your file."
    try:
        key_name = encrypted_name.replace(".enc", ".key")
        encrypted_path = os.path.join(self.ENCRYPTED_DIR, encrypted_name)
        key_path = os.path.join(self.KEYS_DIR, key_name)
        for path in [encrypted_path, key_path]:
            if os.path.exists(path):
                os.remove(path)
        original_name = encrypted_name.replace(f"{username}_", "").replace(".enc", "")
        self.delete_file_metadata(username, original_name)
        self._log_action(username, "DELETE", f"Deleted {encrypted_name}")
        return "Success! File and key deleted."
    except Exception as e:
        return f"Error: {e}"

def delete_key(self, username, key_name):
    if not key_name.startswith(f"{username}_"):
        return "Error: Not your key."
    try:
        key_path = os.path.join(self.KEYS_DIR, key_name)
        if os.path.exists(key_path):
            os.remove(key_path)
            self._log_action(username, "DELETE_KEY", f"Deleted key: {key_name}")
            return "Success! Key deleted."
        return "Error: Key file not found."
    except Exception as e:
        return f"Error: {e}"

def get_recent_activity(self, username, limit=5):
    try:
        if not os.path.exists(self.LOGS_FILE):
            return []
        with open(self.LOGS_FILE, 'r') as f:
            lines = [line.strip() for line in reversed(f.readlines()) if f"USER: {username}" in line]

```

```

        return lines[:limit]
    except Exception:
        return []
def backup_user_keys(self, username):
    try:
        keys = self.get_user_keys(username)
        if not keys:
            return "Error: No keys to backup."
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        backup_path = os.path.join(self.BACKUP_DIR,
f"{username}_keys_backup_{timestamp}.txt")
        with open(backup_path, 'w') as f:
            f.write(f"C-Encrypt Key Backup for {username}\n")
            f.write(f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n")
            f.write("=" * 60 + "\n\n")
            for key_file in keys:
                f.write(f"Key: {key_file}\n")
                with open(os.path.join(self.KEYS_DIR, key_file), 'rb') as kf:
                    f.write(f>Data: {kf.read().decode('utf-8')}\n\n")
            self._log_action(username, "BACKUP_KEYS", f"Backed up {len(keys)} keys")
            return f"Success! Keys backed up to: {backup_path}"
    except Exception as e:
        return f"Error: {e}"
def change_password(self, username, old_password, new_password):
    try:
        lines = []
        found = False
        if old_password:
            old_hash = self._hash_password(old_password)
        else:
            old_hash = None
        new_hash = self._hash_password(new_password)
        with open(self.USERS_FILE, 'r') as f:
            for line in f:
                parts = line.strip().split(':')
                if len(parts) >= 2 and parts[0] == username:
                    if old_hash is None or parts[1] == old_hash:
                        role = parts[2] if len(parts) > 2 else "USER"
                        lines.append(f"{username}:{new_hash}:{role}:false\n")
                        found = True
                    else:
                        return "Error: Current password is incorrect."
                else:
                    lines.append(line)
        if not found:
            return "Error: User not found."
        with open(self.USERS_FILE, 'w') as f:
            f.writelines(lines)
        self._log_action(username, "PASSWORD_CHANGE", "Password changed")
        return "Success"
    except Exception as e:
        return f"Error: {e}"
def get_all_users_detailed(self):
    try:
        users = []
        with open(self.USERS_FILE, 'r') as f:
            for line in f:
                parts = line.strip().split(':')
                if len(parts) >= 2:
                    username = parts[0]
                    role = parts[2] if len(parts) > 2 else "USER"
                    users.append({
                        "username": username,
                        "role": role,
                        "files": len(self.get_user_files(username)),
                        "keys": len(self.get_user_keys(username))
                    })
    
```



```

        return users
    except Exception:
        return []
def delete_user_account(self, admin_user, target_user):
    if target_user == self.ADMIN_USER:
        return "Error: Cannot delete admin account."
    try:
        for f in self.get_user_files(target_user):
            if f.startswith(f"{target_user}_"):
                self.delete_file(target_user, f)
        with open(self.USERS_FILE, 'r') as f:
            lines = [line for line in f if not line.startswith(f"{target_user}:")]
        with open(self.USERS_FILE, 'w') as f:
            f.writelines(lines)
        self._log_action(admin_user, "DELETE_USER", f"Deleted user: {target_user}")
        return f"Success! User '{target_user}' deleted."
    except Exception as e:
        return f"Error: {e}"
def reset_user_password(self, admin_user, target_user, new_password):
    is_strong, issues = self.check_password_strength(new_password)
    if not is_strong:
        return "Password requirements not met:\n" + "\n".join(f" * {issue}" for issue in
issues)
    try:
        new_hash = self._hash_password(new_password)
        lines = []
        found = False
        with open(self.USERS_FILE, 'r') as f:
            for line in f:
                parts = line.strip().split(':')
                if len(parts) >= 2 and parts[0] == target_user:
                    role = parts[2] if len(parts) > 2 else "USER"
                    lines.append(f"{target_user}:{new_hash}:{role}:false\n")
                    found = True
                else:
                    lines.append(line)
        if not found:
            return "Error: User not found."
        with open(self.USERS_FILE, 'w') as f:
            f.writelines(lines)
        self._log_action(admin_user, "RESET_PASSWORD", f"Reset password for:
{target_user}")
        return "Success"
    except Exception as e:
        return f"Error: {e}"
def set_password_reset_required(self, admin_user, target_user, required=True):
    try:
        lines = []
        found = False
        with open(self.USERS_FILE, 'r') as f:
            for line in f:
                parts = line.strip().split(':')
                if len(parts) >= 2 and parts[0] == target_user:
                    role = parts[2] if len(parts) > 2 else "USER"
                    reset_flag = "true" if required else "false"
                    lines.append(f"{target_user}:{parts[1]}:{role}:{reset_flag}\n")
                    found = True
                else:
                    lines.append(line)
        if not found:
            return "Error: User not found."
        with open(self.USERS_FILE, 'w') as f:
            f.writelines(lines)
        action = "enabled" if required else "disabled"
        self._log_action(admin_user, "SET_RESET_FLAG", f"Password reset {action} for:
{target_user}")
        return f"Success! Password reset requirement {action} for '{target_user}'."

```

```

        except Exception as e:
            return f"Error: {e}"
def get_user_activity_summary(self):
    try:
        if not os.path.exists(self.LOGS_FILE):
            return {}
        activity = {}
        with open(self.LOGS_FILE, 'r') as f:
            for line in f:
                if "USER:" in line:
                    parts = line.split("USER:")
                    if len(parts) > 1:
                        user_part = parts[1].split("-")[0].strip()
                        action_part = line.split("ACTION:")[1].split("-")[0].strip() if
"ACTION:" in line else ""
                        if user_part not in activity:
                            activity[user_part] = {"uploads": 0, "downloads": 0,
"deletes": 0, "logins": 0}
                        if "UPLOAD" in action_part:
                            activity[user_part]["uploads"] += 1
                        elif "DOWNLOAD" in action_part and "FAIL" not in action_part:
                            activity[user_part]["downloads"] += 1
                        elif "DELETE" in action_part:
                            activity[user_part]["deletes"] += 1
                        elif action_part == "LOGIN":
                            activity[user_part]["logins"] += 1
                    return activity
        except Exception:
            return {}
def export_metadata(self):
    try:
        if not os.path.exists(self.METADATA_FILE):
            return "Error: No metadata to export."
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        export_path = os.path.join(self.BACKUP_DIR, f"metadata_export_{timestamp}.json")
        shutil.copy(self.METADATA_FILE, export_path)
        return f"Success! Metadata exported to: {export_path}"
    except Exception as e:
        return f"Error: {e}"
def find_orphaned_files(self):
    try:
        with open(self.METADATA_FILE, 'r') as f:
            metadata = json.load(f)
            all_enc_files = set(os.listdir(self.ENCRYPTED_DIR))
            all_key_files = set(os.listdir(self.KEYS_DIR))
            tracked_files = set()
            tracked_keys = set()
            for key in metadata.keys():
                parts = key.split('_', 1)
                if len(parts) == 2:
                    username, filename = parts
                    tracked_files.add(f"{username}_{filename}.enc")
                    tracked_keys.add(f"{username}_{filename}.key")
            orphaned_enc = all_enc_files - tracked_files
            orphaned_keys = all_key_files - tracked_keys
            return list(orphaned_enc), list(orphaned_keys)
    except Exception as e:
        return [], []
def cleanup_orphaned_files(self, admin_user, orphaned_enc, orphaned_keys):
    try:
        removed = []
        for enc_file in orphaned_enc:
            enc_path = os.path.join(self.ENCRYPTED_DIR, enc_file)
            if os.path.exists(enc_path):
                os.remove(enc_path)
                removed.append(enc_file)
        for key_file in orphaned_keys:

```

```

        key_path = os.path.join(self.KEYS_DIR, key_file)
        if os.path.exists(key_path):
            os.remove(key_path)
            removed.append(key_file)
        self._log_action(admin_user, "CLEANUP", f"Removed {len(removed)} orphaned files")
        return f"Success! Removed {len(removed)} orphaned files."
    except Exception as e:
        return f"Error: {e}"

def save_file_metadata(self, username, filename, size, file_hash=None):
    try:
        with open(self.METADATA_FILE, 'r') as f:
            metadata = json.load(f)
        key = f"{username}_{filename}"
        metadata[key] = {
            "upload_time": datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
            "size": size,
            "encrypted": True,
            "hash": file_hash
        }
        with open(self.METADATA_FILE, 'w') as f:
            json.dump(metadata, f, indent=2)
    except Exception:
        pass

def get_file_metadata(self, owner, filename):
    try:
        with open(self.METADATA_FILE, 'r') as f:
            metadata = json.load(f)
        return metadata.get(f"{owner}_{filename}")
    except Exception:
        return None

def delete_file_metadata(self, username, filename):
    try:
        with open(self.METADATA_FILE, 'r') as f:
            metadata = json.load(f)
        key = f"{username}_{filename}"
        if key in metadata:
            del metadata[key]
            with open(self.METADATA_FILE, 'w') as f:
                json.dump(metadata, f, indent=2)
    except Exception:
        pass

def get_logs(self):
    try:
        if not os.path.exists(self.LOGS_FILE):
            return "No logs found."
        with open(self.LOGS_FILE, 'r') as f:
            logs = f.read()
        return logs if logs else "Log file is empty."
    except Exception as e:
        return f"Error: {e}"

def clear_logs(self):
    try:
        open(self.LOGS_FILE, 'w').close()
        return "Logs cleared successfully."
    except Exception as e:
        return f"Error: {e}"

def export_logs(self):
    try:
        if not os.path.exists(self.LOGS_FILE):
            return "Error: No logs to export."
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        export_path = os.path.join(self.BACKUP_DIR, f"logs_export_{timestamp}.txt")
        shutil.copy(self.LOGS_FILE, export_path)
        return f"Success! Logs exported to: {export_path}"
    except Exception as e:
        return f"Error: {e}"

def admin_delete_any_file(self, admin_user, encrypted_name):

```

```

try:
    key_name = encrypted_name.replace(".enc", ".key")
    for path in [os.path.join(self.ENCRYPTED_DIR, encrypted_name),
                  os.path.join(self.KEYS_DIR, key_name)]:
        if os.path.exists(path):
            os.remove(path)
            self._log_action(admin_user, "ADMIN_DELETE", f"Deleted {encrypted_name}")
            return "Success! File deleted."
except Exception as e:
    return f"Error: {e}"
def is_maintenance_mode(self):
    return os.path.exists(self.MAINTENANCE_FILE)
def set_maintenance_mode(self, enabled):
    try:
        if enabled:
            with open(self.MAINTENANCE_FILE, 'w') as f:
                f.write(datetime.now().strftime('%Y-%m-%d %H:%M:%S'))
            return "Maintenance mode enabled."
        else:
            if os.path.exists(self.MAINTENANCE_FILE):
                os.remove(self.MAINTENANCE_FILE)
                return "Maintenance mode disabled."
    except Exception as e:
        return f"Error: {e}"
def clear_backups(self, admin_user):
    try:
        if not os.path.exists(self.BACKUP_DIR):
            return "Error: Backup directory not found."
        files = os.listdir(self.BACKUP_DIR)
        if not files:
            return "No backup files to clear."
        removed_count = 0
        for file in files:
            file_path = os.path.join(self.BACKUP_DIR, file)
            if os.path.isfile(file_path):
                os.remove(file_path)
                removed_count += 1
        self._log_action(admin_user, "CLEAR_BACKUPS", f"Cleared {removed_count} backup
files")
        return f"Success! Cleared {removed_count} backup file(s)."
```

```

    except Exception as e:
        return f"Error: {e}"
def delete_all_files_and_keys(self, admin_user):
    try:
        enc_files = os.listdir(self.ENCRYPTED_DIR) if os.path.exists(self.ENCRYPTED_DIR)
    else []
    key_files = os.listdir(self.KEYS_DIR) if os.path.exists(self.KEYS_DIR) else []
    if not enc_files and not key_files:
        return "No files or keys to delete."
    removed_count = 0
    for file in enc_files:
        file_path = os.path.join(self.ENCRYPTED_DIR, file)
        if os.path.isfile(file_path):
            os.remove(file_path)
            removed_count += 1
    for file in key_files:
        file_path = os.path.join(self.KEYS_DIR, file)
        if os.path.isfile(file_path):
            os.remove(file_path)
            removed_count += 1
    with open(self.METADATA_FILE, 'w') as f:
        json.dump({}, f)
    self._log_action(admin_user, "DELETE_ALL_FILES", f"Deleted all files and keys
({removed_count} items)")
    return f"Success! Deleted {len(enc_files)} file(s) and {len(key_files)} key(s)."
```

```

    except Exception as e:
        return f"Error: {e}"

```

```

def delete_all_user_data(self, username, password):
    hashed_password = self._hash_password(password)
    with open(self.USERS_FILE, 'r') as f:
        for line in f:
            parts = line.strip().split(':')
            if len(parts) >= 2 and parts[0] == username:
                if parts[1] != hashed_password:
                    return "Error: Incorrect password."
                break
    try:
        deleted_count = {"files": 0, "keys": 0, "backups": 0}
        for file in os.listdir(self.ENCRYPTED_DIR):
            if file.startswith(f"{username}_"):
                os.remove(os.path.join(self.ENCRYPTED_DIR, file))
                deleted_count["files"] += 1
        for key in os.listdir(self.KEYS_DIR):
            if key.startswith(f"{username}_"):
                os.remove(os.path.join(self.KEYS_DIR, key))
                deleted_count["keys"] += 1
        for backup in os.listdir(self.BACKUP_DIR):
            if backup.startswith(f"{username}_"):
                os.remove(os.path.join(self.BACKUP_DIR, backup))
                deleted_count["backups"] += 1
        with open(self.METADATA_FILE, 'r') as f:
            metadata = json.load(f)
        keys_to_delete = [k for k in metadata.keys() if k.startswith(f"{username}_")]
        for key in keys_to_delete:
            del metadata[key]
        with open(self.METADATA_FILE, 'w') as f:
            json.dump(metadata, f, indent=2)
        self._log_action(username, "DELETE_ALL_DATA",
            f"Deleted {deleted_count['files']} files, {deleted_count['keys']}
keys, {deleted_count['backups']} backups")
        return f"Success! Deleted {deleted_count['files']} file(s),
{deleted_count['keys']} key(s), and {deleted_count['backups']} backup(s)."
```

```

    except Exception as e:
        return f"Error: {e}"

def delete_own_account(self, username, password):
    if username == self.ADMIN_USER:
        return "Error: Cannot delete admin account."
    hashed_password = self._hash_password(password)
    with open(self.USERS_FILE, 'r') as f:
        for line in f:
            parts = line.strip().split(':')
            if len(parts) >= 2 and parts[0] == username:
                if parts[1] != hashed_password:
                    return "Error: Incorrect password."
                break
    data_result = self.delete_all_user_data(username, password)
    if "Error" in data_result:
        return data_result
    try:
        with open(self.USERS_FILE, 'r') as f:
            lines = [line for line in f if not line.startswith(f"{username}:")]
        with open(self.USERS_FILE, 'w') as f:
            f.writelines(lines)
        self._log_action(username, "DELETE_OWN_ACCOUNT", "User deleted their own account")
        return "Success! Your account and all data have been permanently deleted."
    except Exception as e:
        return f"Error: {e}"

def _format_file_size(self, size_bytes):
    if size_bytes < 1024:
        return f"{size_bytes} bytes"
    elif size_bytes < 1024 * 1024:
        return f"{size_bytes / 1024:.2f} KB"
    elif size_bytes < 1024 * 1024 * 1024:
        return f"{size_bytes / (1024 * 1024):.2f} MB"
```

[illegible]

```

        elif choice == '2':
            self.register_flow()
        elif choice == '3':
            self.clear_screen()
            print("\n Thank you for using C-Encrypt!")
            print("\n Goodbye!\n")
            sys.exit(0)
        else:
            print("\n [ERROR] Invalid choice. Try again.")
            self.pause()
def register_flow(self):
    self.clear_screen()
    self.print_header("USER REGISTRATION")
    print("\n Password Requirements:\n")
    print("    * Minimum 8 characters")
    print("    * At least one uppercase letter")
    print("    * At least one number\n")
    username = self.safe_input("Username: ")
    if not username:
        return
    password = self.safe_input("Password: ")
    if not password:
        print("\n [ERROR] Password cannot be empty.")
        self.pause()
        return
    confirm = self.safe_input("Confirm Password: ")
    if password != confirm:
        print("\n [ERROR] Passwords don't match.")
        self.pause()
        return
    result = self.logic.register_user(username, password)
    if result == "Success":
        print("\n [SUCCESS] Registration complete!")
        print("\n Logging you in...")
        self.pause()
        success, role, error_msg, reset_required = self.logic.login_user(username,
password)
        if success:
            self.current_user = username
            self.user_role = role
            self.clear_screen()
            print(f"\n [SUCCESS] Welcome, {self.current_user}!")
            self.pause()
            if role == "ADMIN":
                self.admin_menu()
            else:
                self.session_menu()
        else:
            print("\n [ERROR] Auto-login failed. Please login manually.")
            self.pause()
    else:
        print(f"\n [ERROR] {result}")
        self.pause()
def login_flow(self):
    self.clear_screen()
    self.print_header("USER LOGIN")
    print()
    username = self.safe_input("Username: ")
    if not username:
        return
    if not self.logic._check_user_exists(username):
        print(f"\n [ERROR] User not found")
        self.pause()
        return
    password = self.safe_password("Password: ")
    success, role, error_msg, reset_required = self.logic.login_user(username, password)
    if success:

```

```

        self.current_user = username
        self.user_role = role
        print(f"\n [SUCCESS] Welcome, {self.current_user}!")
        if role == "ADMIN":
            print(" [ADMIN] Administrator access granted")
            self.pause()
            if reset_required:
                self.forced_password_reset()
            if role == "ADMIN":
                self.admin_menu()
            else:
                self.session_menu()
        else:
            print(f"\n [ERROR] {error_msg if error_msg else 'Invalid credentials.'}")
            self.pause()
def forced_password_reset(self):
    self.clear_screen()
    self.print_header("PASSWORD RESET REQUIRED")
    print("\n Your administrator has required you to reset your password.\n")
    new_password = self.safe_password("New Password: ")
    if not new_password:
        print("\n [ERROR] Password cannot be empty.")
        self.pause()
        sys.exit(0)
    is_strong, issues = self.logic.check_password_strength(new_password)
    if not is_strong:
        print("\n [ERROR] Password requirements not met:")
        for issue in issues:
            print(f" * {issue}")
        self.pause()
        sys.exit(0)
    confirm = self.safe_password("Confirm New Password: ")
    if new_password != confirm:
        print("\n [ERROR] Passwords don't match.")
        self.pause()
        sys.exit(0)
    result = self.logic.change_password(self.current_user, "", new_password)
    if "Success" in result or result == "Success":
        print("\n [SUCCESS] Password changed successfully!")
        self.pause()
    else:
        print(f"\n [ERROR] Failed to change password. Contact administrator.")
        self.pause()
        sys.exit(0)
def session_menu(self):
    while True:
        self.clear_screen()
        print(self.main_banner)
        self.print_header(f"USER DASHBOARD - {self.current_user}")
        print("\n FILE OPERATIONS ACCOUNT & SECURITY DANGER!\n")
        print(" [1] Upload File [7] List My Keys [13] Delete All")
        print(" [2] Download File [8] Delete a Key [14] Delete My")
        print(" [3] List My Files [9] Backup Keys")
        print(" [4] File Info [10] Change Password")
        print(" [5] Share a Key [11] Recent Activity")
        print(" [6] Delete a File [12] Logout\n")
        choice = self.safe_input("Select option: ")
        if choice == '1':
            self.upload_flow()
        elif choice == '2':
            self.download_flow()
        elif choice == '3':
            self.list_files_flow()
        elif choice == '4':
            self.file_info_flow()

```



```

elif choice == '5':
    self.share_flow()
elif choice == '6':
    self.delete_file_flow()
elif choice == '7':
    self.list_keys_flow()
elif choice == '8':
    self.delete_key_flow()
elif choice == '9':
    self.backup_keys_flow()
elif choice == '10':
    self.change_password_flow()
elif choice == '11':
    self.recent_activity_flow()
elif choice == '12':
    self.logic._log_action(self.current_user, "LOGOUT", "User logged out")
    self.current_user = None
    self.user_role = None
    print("\n [SUCCESS] Logged out.")
    self.pause()
    break
elif choice == '13':
    self.delete_all_data_flow()
elif choice == '14':
    self.delete_own_account_flow()
else:
    print("\n [ERROR] Invalid choice. Try again.")
    self.pause()
def admin_menu(self):
    while True:
        self.clear_screen()
        self.print_header(f"ADMIN PANEL - {self.current_user}")
        print("\n  USER MANAGEMENT                FILE & LOG MANAGEMENT\n")
        print("    [1] View All Users                [5] Delete Any File")
        print("    [2] Delete User Account          [6] View All Logs")
        print("    [3] Force Password Reset         [7] Export Logs")
        print("    [4] View All Files                [8] Clear All Logs")
        print("                                     [9] Clear All Backups")
        print("                                     [10] Delete All Files & Keys")
        print("\n  SYSTEM ADMINISTRATION\n")
        print("    [11] System Statistics            [14] System Cleanup Utility")
        print("    [12] User Activity Summary        [15] Maintenance Mode")
        print("    [13] Export Metadata              [16] Logout\n")
        choice = self.safe_input("Select option: ")
        if choice == '1':
            self.view_all_users_detailed()
        elif choice == '2':
            self.delete_user_flow()
        elif choice == '3':
            self.force_reset_flow()
        elif choice == '4':
            self.view_all_files()
        elif choice == '5':
            self.admin_delete_file_flow()
        elif choice == '6':
            self.logs_flow()
        elif choice == '7':
            self.export_logs_flow()
        elif choice == '8':
            self.clear_logs_flow()
        elif choice == '9':
            self.clear_backups_flow()
        elif choice == '10':
            self.delete_all_files_and_keys_flow()
        elif choice == '11':
            self.system_stats()
        elif choice == '12':

```

```

        self.user_activity_summary_flow()
    elif choice == '13':
        self.export_metadata_flow()
    elif choice == '14':
        self.cleanup_utility_flow()
    elif choice == '15':
        self.maintenance_mode_flow()
    elif choice == '16':
        self.logic._log_action(self.current_user, "LOGOUT", "Admin logged out")
        self.current_user = None
        self.user_role = None
        print("\n [SUCCESS] Logged out.")
        self.pause()
        break
    else:
        print("\n [ERROR] Invalid choice. Try again.")
        self.pause()
def _show_progress_bar(self, message, duration=2.0):
    print(f"\n {message}")
    bar_length = 50
    for i in range(bar_length + 1):
        percent = (i / bar_length) * 100
        filled = '█' * i
        empty = '░' * (bar_length - i)
        print(f"\r [{filled}{empty}] {percent:.0f}%", end='', flush=True)
        time.sleep(duration / bar_length)
    print()
def upload_flow(self):
    self.clear_screen()
    self.print_header("UPLOAD FILE")
    if self.logic.is_maintenance_mode():
        print("\n Uploads are currently disabled during maintenance.")
        self.pause()
        return
    print("\n Enter the file path (with or without quotes)\n")
    file_path = self.safe_input("File Path: ")
    if not file_path:
        return
    file_path = self.strip_quotes(file_path)
    self._show_progress_bar("Encrypting and uploading file...", 1.5)
    result = self.logic.upload_file(self.current_user, file_path)
    if "Success" in result:
        print(f"\n [SUCCESS] {result}")
    else:
        print(f"\n [ERROR] {result}")
    self.pause()
def list_files_flow(self):
    self.clear_screen()
    self.print_header("YOUR ENCRYPTED FILES")
    files = self.logic.get_user_files(self.current_user)
    print()
    if not files:
        print(" No files found.")
    else:
        for i, f in enumerate(files, 1):
            display_name = self._format_display_name(f)
            tag = " (shared)" if not f.startswith(f"{self.current_user}_") else ""
            print(f" [{i}] {display_name}{tag}")
        self.pause()
def file_info_flow(self):
    self.clear_screen()
    self.print_header("FILE INFORMATION")
    files = self.logic.get_user_files(self.current_user)
    if not files:
        print("\n No files available.")
        self.pause()
        return

```

```

print("\n SELECT FILE\n")
for i, f in enumerate(files, 1):
    print(f" [{i}] {self._format_display_name(f)}")
print()
idx = self._get_valid_index(f"Select file (1-{len(files)}): ", len(files))
if idx is None:
    return
selected = files[idx]
original_name = self._format_display_name(selected)
owner = selected.split('_')[0] if '_' in selected else self.current_user
metadata_key = selected.replace('.enc', '')
parts = metadata_key.split('_', 1)
if len(parts) == 2:
    owner, base_name = parts
    with open(self.logic.METADATA_FILE, 'r') as f:
        all_metadata = json.load(f)
        metadata = all_metadata.get(f"{owner}_{base_name}")
else:
    metadata = None
print(f"\n File: {original_name}")
if metadata:
    print(f" Upload Time: {metadata.get('upload_time', 'Unknown')}")
    size_bytes = metadata.get('size', 0)
    print(f" Size: {self.logic._format_file_size(size_bytes)}")
    print(f" Encrypted: Yes")
    if metadata.get('hash'):
        print(f" Integrity: Protected")
else:
    print(" No metadata available.")
self.pause()
def _get_valid_index(self, prompt, max_index):
    choice = self.safe_input(prompt)
    if not choice:
        return None
    try:
        idx = int(choice) - 1
        if 0 <= idx < max_index:
            return idx
        raise ValueError()
    except (ValueError, IndexError):
        print("\n [ERROR] Invalid selection. Try again.")
        self.pause()
        return None
def download_flow(self):
    self.clear_screen()
    self.print_header("DOWNLOAD & DECRYPT FILE")
    files = self.logic.get_user_files(self.current_user)
    file_idx = self._select_from_list(files, "file")
    if file_idx is None:
        return
    keys = self.logic.get_user_keys(self.current_user)
    key_idx = self._select_from_list(keys, "key")
    if key_idx is None:
        return
    self._show_progress_bar("Downloading and decrypting file...", 1.5)
    key_path = os.path.join(self.logic.KEYS_DIR, keys[key_idx])
    result = self.logic.download_file(self.current_user, files[file_idx], key_path)
    self._display_result(result)
    self.pause()
def share_flow(self):
    self.clear_screen()
    self.print_header("SHARE ENCRYPTION KEY")
    keys = self.logic.get_user_keys(self.current_user)
    idx = self._select_from_list(keys, "key")
    if idx is None:
        return
    print()

```

```

        target = self.safe_input("Share with username: ")
        if not target:
            return
        print("\n Processing...")
        result = self.logic.share_key(self.current_user, keys[idx], target)
        self._display_result(result)
        self.pause()
def delete_file_flow(self):
    self.clear_screen()
    self.print_header("DELETE FILE")
    files = self.logic.get_user_files(self.current_user)
    own_files = [f for f in files if f.startswith(f"{self.current_user}_")]
    idx = self._select_from_list(own_files, "file")
    if idx is None:
        return
    display_name = self._format_display_name(own_files[idx])
    if self._confirm_action(f"Delete '{display_name}' permanently?"):
        result = self.logic.delete_file(self.current_user, own_files[idx])
        self._display_result(result)
    else:
        print("\n [INFO] Deletion cancelled.")
        self.pause()
def delete_key_flow(self):
    self.clear_screen()
    self.print_header("DELETE ENCRYPTION KEY")
    keys = self.logic.get_user_keys(self.current_user)
    idx = self._select_from_list(keys, "key")
    if idx is None:
        return
    display_name = self._format_display_name(keys[idx])
    print(f"\n WARNING: Delete key '{display_name}' permanently?")
    print(" You won't be able to decrypt the file without this key.\n")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.delete_key(self.current_user, keys[idx])
        self._display_result(result)
    else:
        print("\n [INFO] Deletion cancelled.")
        self.pause()
def delete_user_flow(self):
    self.clear_screen()
    self.print_header("DELETE USER ACCOUNT")
    print("\n WARNING: This will permanently delete the user and all data.\n")
    target = self.safe_input("Enter username to delete: ")
    if not target:
        return
    print(f"\n Confirm deletion of user '{target}'")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.delete_user_account(self.current_user, target)
        self._display_result(result)
    else:
        print("\n [INFO] Deletion cancelled.")
        self.pause()
def admin_delete_file_flow(self):
    self.clear_screen()
    self.print_header("DELETE ANY FILE")
    files = self.logic.get_all_files()
    idx = self._select_from_list(files, "file")
    if idx is None:
        return
    selected = files[idx]
    print(f"\n Delete '{selected}'?")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.admin_delete_any_file(self.current_user, selected)
        self._display_result(result)
    else:
        print("\n [INFO] Deletion cancelled.")
        self.pause()

```

```

def export_logs_flow(self):
    self.clear_screen()
    self.print_header("EXPORT LOGS")
    print("\n Exporting logs...\n")
    result = self.logic.export_logs()
    self._display_result(result)
    self.pause()

def export_metadata_flow(self):
    self.clear_screen()
    self.print_header("EXPORT METADATA")
    print("\n Exporting metadata...\n")
    result = self.logic.export_metadata()
    self._display_result(result)
    self.pause()

def reset_password_flow(self):
    self.clear_screen()
    self.print_header("RESET USER PASSWORD")
    print("\n WARNING: This will permanently reset the user's password.\n")
    target = self.safe_input("Enter username to reset: ")
    if not target:
        return
    if not self.logic._check_user_exists(target):
        print(f"\n [ERROR] User '{target}' not found.")
        self.pause()
        return
    if target == self.logic.ADMIN_USER:
        print(f"\n [ERROR] Cannot reset admin password.")
        self.pause()
        return
    print("\n Password Requirements:")
    print("    * Minimum 8 characters")
    print("    * At least one uppercase letter")
    print("    * At least one number\n")
    new_password = self.safe_password("New Password: ")
    if not new_password:
        return
    result = self.logic.reset_user_password(self.current_user, target, new_password)
    if result == "Success":
        print(f"\n [SUCCESS] Password reset for '{target}'!")
        print(f"    New Password: {new_password}")
    else:
        self._display_result(result)
    self.pause()

def delete_all_data_flow(self):
    self.clear_screen()
    self.print_header("DELETE ALL MY DATA")
    print("\n CRITICAL WARNING")
    print("\n This will permanently delete:")
    print("    - ALL your encrypted files")
    print("    - ALL your encryption keys (owned and shared)")
    print("    - ALL your backups")
    print("\n This action CANNOT be undone!")
    print("    Your account will remain active but all data will be lost.\n")
    confirm1 = self.safe_input("Type 'DELETE MY DATA' to confirm: ")
    if confirm1 != "DELETE MY DATA":
        print("\n [INFO] Operation cancelled.")
        self.pause()
        return
    password = self.safe_password("\nEnter your password to confirm: ")
    if not password:
        print("\n [INFO] Operation cancelled.")
        self.pause()
        return
    result = self.logic.delete_all_user_data(self.current_user, password)
    self._display_result(result)
    self.pause()

def delete_own_account_flow(self):

```

```

self.clear_screen()
self.print_header("DELETE MY ACCOUNT")
print("\n  CRITICAL WARNING")
print("\n  This will permanently:")
print("    - DELETE your account")
print("    - DELETE all your files and keys")
print("    - DELETE all your backups")
print("    - LOG YOU OUT immediately")
print("\n  This action is IRREVERSIBLE!")
print("  You will need to register again to use this system.\n")
confirm1 = self.safe_input("Type 'DELETE MY ACCOUNT' to confirm: ")
if confirm1 != "DELETE MY ACCOUNT":
    print("\n  [INFO] Operation cancelled.")
    self.pause()
    return
password = self.safe_password("\nEnter your password to confirm: ")
if not password:
    print("\n  [INFO] Operation cancelled.")
    self.pause()
    return
confirm2 = self.safe_input("\nAre you absolutely sure? (yes/no): ")
if confirm2.lower() != "yes":
    print("\n  [INFO] Operation cancelled.")
    self.pause()
    return
result = self.logic.delete_own_account(self.current_user, password)
if "Success" in result:
    print(f"\n  [SUCCESS] {result}")
    print("\n  You will now be logged out.")
    self.pause()
    self.current_user = None
    self.user_role = None
else:
    self._display_result(result)
    self.pause()
def list_keys_flow(self):
    self.clear_screen()
    self.print_header("YOUR ENCRYPTION KEYS")
    keys = self.logic.get_user_keys(self.current_user)
    print()
    if not keys:
        print("  No keys found.")
    else:
        for i, k in enumerate(keys, 1):
            display_name = self._format_display_name(k)
            tag = " (shared)" if not k.startswith(f"{self.current_user}_") else ""
            print(f"  [{i}] {display_name}{tag}")
        self.pause()
def backup_keys_flow(self):
    self.clear_screen()
    self.print_header("BACKUP ENCRYPTION KEYS")
    print("\n  This will export all your keys to a backup file.\n")
    confirm = self.safe_input("Continue? (y/n): ")
    if confirm.lower() == 'y':
        result = self.logic.backup_user_keys(self.current_user)
        self._display_result(result)
    else:
        print("\n  [INFO] Backup cancelled.")
        self.pause()
def change_password_flow(self):
    self.clear_screen()
    self.print_header("CHANGE PASSWORD")
    print()
    old_password = self.safe_password("Current Password: ")
    if not old_password:
        return
    new_password = self.safe_password("New Password: ")

```

```

        if not new_password:
            return
        is_strong, issues = self.logic.check_password_strength(new_password)
        if not is_strong:
            print("\n [ERROR] Password requirements not met:")
            for issue in issues:
                print(f"      * {issue}")
            self.pause()
            return
        confirm = self.safe_password("Confirm New Password: ")
        if new_password != confirm:
            print("\n [ERROR] Passwords don't match.")
            self.pause()
            return
        result = self.logic.change_password(self.current_user, old_password, new_password)
        if result == "Success":
            print("\n [SUCCESS] Password changed successfully!")
        else:
            self._display_result(result)
            self.pause()
    def recent_activity_flow(self):
        self.clear_screen()
        self.print_header("RECENT ACTIVITY")
        activities = self.logic.get_recent_activity(self.current_user)
        print()
        if not activities:
            print(" No recent activity.")
        else:
            for activity in activities:
                print(f" {activity}")
            self.pause()
    def view_all_users_detailed(self):
        self.clear_screen()
        self.print_header("ALL USERS - DETAILED VIEW")
        users = self.logic.get_all_users_detailed()
        print()
        for user in users:
            print(f" Username: {user['username']}")
            print(f" Role: {user['role']}")
            print(f" Files: {user['files']} | Keys: {user['keys']}")
            print()
        self.pause()
    def force_reset_flow(self):
        self.clear_screen()
        self.print_header("FORCE PASSWORD RESET")
        print("\n This will immediately reset the user's password.\n")
        target = self.safe_input("Enter username: ")
        if not target:
            return
        if not self.logic._check_user_exists(target):
            print(f"\n [ERROR] User '{target}' not found.")
            self.pause()
            return
        if target == self.logic.ADMIN_USER:
            print(f"\n [ERROR] Cannot reset admin password.")
            self.pause()
            return
        print("\n Password Requirements:")
        print("      * Minimum 8 characters")
        print("      * At least one uppercase letter")
        print("      * At least one number\n")
        new_password = self.safe_password("New Password: ")
        if not new_password:
            return
        result = self.logic.reset_user_password(self.current_user, target, new_password)
        if result == "Success":
            print(f"\n [SUCCESS] Password reset for '{target}'!")

```

```

        print(f"  New Password: {new_password}")
    else:
        self._display_result(result)
        self.pause()
def view_all_files(self):
    self.clear_screen()
    self.print_header("ALL ENCRYPTED FILES")
    files = self.logic.get_all_files()
    print()
    if not files:
        print("  No files in storage.")
    else:
        for f in files:
            print(f"    * {f}")
        self.pause()
def logs_flow(self):
    self.clear_screen()
    self.print_header("SYSTEM LOGS")
    logs = self.logic.get_logs()
    print("\n" + logs)
    self.pause()
def clear_logs_flow(self):
    self.clear_screen()
    self.print_header("CLEAR LOGS")
    print("\n  WARNING: This action is permanent.\n")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.clear_logs()
        print(f"\n    [SUCCESS] {result}")
        self.logic._log_action(self.current_user, "CLEAR_LOGS", "All logs cleared")
    else:
        print("\n    [INFO] Action cancelled.")
    self.pause()
def clear_backups_flow(self):
    self.clear_screen()
    self.print_header("CLEAR ALL BACKUPS")
    print("\n  WARNING: This will permanently delete all backup files.\n")
    print("  This includes:")
    print("    - Key backups")
    print("    - Log exports")
    print("    - Metadata exports\n")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.clear_backups(self.current_user)
        self._display_result(result)
    else:
        print("\n    [INFO] Action cancelled.")
    self.pause()
def delete_all_files_and_keys_flow(self):
    self.clear_screen()
    self.print_header("DELETE ALL FILES & KEYS")
    print("\n  ⚠ CRITICAL WARNING ⚠")
    print("\n  This will permanently delete:")
    print("    - ALL encrypted files")
    print("    - ALL encryption keys")
    print("    - ALL file metadata")
    print("\n  This action CANNOT be undone!")
    print("  All users will lose access to their files.\n")
    confirm1 = self.safe_input("Type 'DELETE ALL' to confirm: ")
    if confirm1 != "DELETE ALL":
        print("\n    [INFO] Operation cancelled.")
        self.pause()
        return
    confirm2 = self.safe_input("Are you absolutely sure? (yes/no): ")
    if confirm2.lower() != "yes":
        print("\n    [INFO] Operation cancelled.")
        self.pause()
        return

```



```

        result = self.logic.delete_all_files_and_keys(self.current_user)
        self._display_result(result)
        self.pause()
def system_stats(self):
    self.clear_screen()
    self.print_header("SYSTEM STATISTICS")
    print()
    users = self.logic.get_all_users()
    files = self.logic.get_all_files()
    keys = os.listdir(self.logic.KEYS_DIR) if os.path.exists(self.logic.KEYS_DIR) else []
    print(f"    Total Users:          {len(users)}")
    print(f"    Total Files:          {len(files)}")
    print(f"    Total Keys:           {len(keys)}")
    print()
    print(f"    Encrypted Path:       {self.logic.ENCRYPTED_DIR}")
    print(f"    Decrypted Path:       {self.logic.DECRYPTED_DIR}")
    self.pause()
def user_activity_summary_flow(self):
    self.clear_screen()
    self.print_header("USER ACTIVITY SUMMARY")
    activity = self.logic.get_user_activity_summary()
    print()
    if not activity:
        print("    No activity data available.")
    else:
        for username, stats in activity.items():
            print(f"    User: {username}")
            print(f"        Logins: {stats['logins']} | Uploads: {stats['uploads']}")
            print(f"        Downloads: {stats['downloads']} | Deletes: {stats['deletes']}")
            print()
        self.pause()
def cleanup_utility_flow(self):
    self.clear_screen()
    self.print_header("SYSTEM CLEANUP UTILITY")
    print("\n    Scanning for orphaned files...\n")
    orphaned_enc, orphaned_keys = self.logic.find_orphaned_files()
    total = len(orphaned_enc) + len(orphaned_keys)
    if total == 0:
        print("    No orphaned files found. System is clean.")
        self.pause()
        return
    print(f"    Found {len(orphaned_enc)} orphaned encrypted files")
    print(f"    Found {len(orphaned_keys)} orphaned key files")
    print(f"\n    Total: {total} orphaned files\n")
    if orphaned_enc:
        print("    Encrypted files:")
        for f in orphaned_enc[:10]:
            print(f"        - {f}")
        if len(orphaned_enc) > 10:
            print(f"        ... and {len(orphaned_enc) - 10} more")
    if orphaned_keys:
        print("\n    Key files:")
        for k in orphaned_keys[:10]:
            print(f"        - {k}")
        if len(orphaned_keys) > 10:
            print(f"        ... and {len(orphaned_keys) - 10} more")
    print("\n    WARNING: This will permanently delete these files.\n")
    if self.safe_input("Type 'y' to confirm: ").lower() == 'y':
        result = self.logic.cleanup_orphaned_files(self.current_user, orphaned_enc,
orphaned_keys)
        self._display_result(result)
    else:
        print("\n    [INFO] Cleanup cancelled.")
        self.pause()
def maintenance_mode_flow(self):
    self.clear_screen()
    self.print_header("SYSTEM MAINTENANCE MODE")

```

```

is_active = self.logic.is_maintenance_mode()
status = "ENABLED" if is_active else "DISABLED"
print(f"\n Current Status: {status}\n")
print(" [1] Enable Maintenance Mode")
print(" [2] Disable Maintenance Mode")
print(" [3] Cancel\n")
choice = self.safe_input("Select option: ")
if choice == '1':
    result = self.logic.set_maintenance_mode(True)
    print(f"\n [SUCCESS] {result}")
elif choice == '2':
    result = self.logic.set_maintenance_mode(False)
    print(f"\n [SUCCESS] {result}")
else:
    print("\n [INFO] No changes made.")
    self.pause()
def _display_result(self, result, success_key="Success"):
    if success_key in result:
        print(f"\n [SUCCESS] {result}")
    else:
        print(f"\n [ERROR] {result}")
def _confirm_action(self, message, confirm_word="y"):
    print(f"\n WARNING: {message}")
    print(" This action cannot be undone.\n")
    return self.safe_input(f"Type '{confirm_word}' to confirm: ").lower() ==
confirm_word.lower()
def _select_from_list(self, items, item_type="item"):
    if not items:
        print(f"\n No {item_type}s available.")
        self.pause()
        return None
    print(f"\n SELECT {item_type.upper()}\n")
    for i, item in enumerate(items, 1):
        display = self._format_display_name(item) if item_type in ["file", "key"] else
item
        if item_type in ["file", "key"]:
            tag = " (shared)" if not item.startswith(f"{self.current_user}_") else ""
        else:
            tag = ""
        print(f" [{i}] {display}{tag}")
    print()
    return self._get_valid_index(f"Select {item_type} (1-{len(items)}): ", len(items))

if __name__ == "__main__":
    try:
        app = ConsoleApp()
        app.run()
    except KeyboardInterrupt:
        print("\n\n Program interrupted.")
        sys.exit(0)
    except Exception as e:
        print(f"\n Fatal error: {e}")
        sys.exit(1)

```

7. Output:

Program Main Menu –

```
=====
                        _ _ _ _ _
                       / /   \ \
                      / /     \ \
                     / /       \ \
                    / /         \ \
                   / /           \ \
                  / /             \ \
                 / /               \ \
                / /                 \ \
               / /                   \ \
              / /                     \ \
             / /                       \ \
            / /                         \ \
           / /                           \ \
          / /                             \ \
         / /                               \ \
        / /                                 \ \
       / /                                   \ \
      / /                                     \ \
     / /                                       \ \
    / /                                         \ \
   / /                                           \ \
  / /                                             \ \
 / /                                               \ \
/ /                                                 \ \
Secure Cloud Platform
=====

Storage: C:\Users\Superduash\Downloads\Cloud encryption\cstorage

[1] Login
[2] Register New User
[3] Exit Application

Select option:
```

New User Registration –

```
USER REGISTRATION
-----

Password Requirements:

* Minimum 8 characters
* At least one uppercase letter
* At least one number

Username: NewUser
Password: User1234
Confirm Password: User1234

[SUCCESS] Registration complete!

Logging you in...
```

User Login –

```
USER LOGIN
-----

Username: Superduash
Password:

[SUCCESS] Welcome, Superduash!
```

User Operations –

Dashboard –

```
=====
          _____
         /  _  \  /  _  \  /  _  \  /  _  \
        /  _  \ /  _  \ /  _  \ /  _  \
       /  _  \ /  _  \ /  _  \ /  _  \
      /  _  \ /  _  \ /  _  \ /  _  \
     /  _  \ /  _  \ /  _  \ /  _  \
    /  _  \ /  _  \ /  _  \ /  _  \
   /  _  \ /  _  \ /  _  \ /  _  \
  /  _  \ /  _  \ /  _  \ /  _  \
 /  _  \ /  _  \ /  _  \ /  _  \
/  _  \ /  _  \ /  _  \ /  _  \
Secure Cloud Platform
=====

USER DASHBOARD - Superduash
-----

FILE OPERATIONS                ACCOUNT & SECURITY                DANGER!

[1] Upload File                [7] List My Keys                [13] Delete All
My Data                        [8] Delete a Key                [14] Delete My
[2] Download File              Account                          [9] Backup Keys
[3] List My Files              [10] Change Password
[4] File Info                  [11] Recent Activity
[5] Share a Key                [12] Logout
[6] Delete a File

Select option:
```

File Upload & Encryption –

```
UPLOAD FILE
-----

Enter the file path (with or without quotes)

File Path: sample.txt

Encrypting and uploading file...
[████████████████████████████████████████] 100%

[SUCCESS] Success! File uploaded as 'Superduash_sample (1).txt.enc'.
```

File Download & Decryption –

```
DOWNLOAD & DECRYPT FILE
-----
```


Select file (1-6): 4

File: sample.mp4

Upload Time: 2025-11-07 06:26:58

Size: 2.24 MB

Encrypted: Yes

Integrity: Protected

Share Key –

SHARE ENCRYPTION KEY

SELECT KEY

[1] logo (1).png

[2] logo.png

[3] sample (1).txt

[4] sample.mp4

[5] sample.py

[6] sample.txt

Select key (1-6): 2

Share with username: Faheem

Processing...

[SUCCESS] Success! Key shared with Faheem.

Delete File –

DELETE FILE

SELECT FILE

[1] logo (1).png

[2] logo.png

[3] sample (1).txt

[4] sample.mp4

[5] sample.txt

Select file (1-5): 1

WARNING: Delete 'logo (1).png' permanently?

This action cannot be undone.

Type 'y' to confirm: y

[SUCCESS] Success! File and key deleted.

List Keys –

YOUR ENCRYPTION KEYS

- [1] logo.png
- [2] sample (1).txt
- [3] sample.mp4
- [4] sample.py
- [5] sample.txt

Delete Keys –

DELETE ENCRYPTION KEY

SELECT KEY

- [1] logo.png
- [2] sample (1).txt
- [3] sample.mp4
- [4] sample.py
- [5] sample.txt

Select key (1-5): 2

WARNING: Delete key 'sample (1).txt' permanently?
You won't be able to decrypt the file without this key.

Type 'y' to confirm: y

[SUCCESS] Success! Key deleted.

Backup Keys –

BACKUP ENCRYPTION KEYS

This will export all your keys to a backup file.

Continue? (y/n): y

[SUCCESS] Success! Keys backed up to: C:\Users\Superduash\Downloads\Cloud encryption\cstorage\backups\Superduash_keys_backup_20251107_080606.txt

Change Password –

CHANGE PASSWORD

Current Password:

New Password:

Confirm New Password:

[SUCCESS] Password changed successfully!

Recent Activity –

RECENT ACTIVITY

[2025-11-07 08:06:36] - USER: Superduash - ACTION: PASSWORD_CHANGE -
DETAILS: Password changed

[2025-11-07 08:06:06] - USER: Superduash - ACTION: BACKUP_KEYS - DETAILS:
Backed up 4 keys

[2025-11-07 08:05:32] - USER: Superduash - ACTION: DELETE_KEY - DETAILS:
Deleted key: Superduash_sample (1).txt.key

[2025-11-07 08:03:49] - USER: Superduash - ACTION: DELETE - DETAILS:
Deleted Superduash_logo (1).png.enc

[2025-11-07 08:00:10] - USER: Superduash - ACTION: SHARE - DETAILS:
Shared with Faheem

Delete Data –

DELETE ALL MY DATA

CRITICAL WARNING

This will permanently delete:

- ALL your encrypted files
- ALL your encryption keys (owned and shared)
- ALL your backups

This action CANNOT be undone!

Your account will remain active but all data will be lost.

Type 'DELETE MY DATA' to confirm: DELETE MY DATA

Enter your password to confirm:

[SUCCESS] Success! Deleted 0 file(s), 0 key(s), and 0 backup(s).

Delete Account –

Type 'DELETE MY ACCOUNT' to confirm: DELETE MY ACCOUNT

Enter your password to confirm:

Are you absolutely sure? (yes/no): yes

[SUCCESS] Success! Your account and all data have been permanently deleted.

You will now be logged out.

Logout –

Select option: 12

[SUCCESS] Logged out.

Admin Interface –

View All Users –

ALL USERS - DETAILED VIEW

Username: Admin
Role: ADMIN
Files: 0 | Keys: 0

Username: Superduash
Role: USER
Files: 5 | Keys: 4

Username: Faheem
Role: USER
Files: 2 | Keys: 2

Username: Harsh
Role: USER
Files: 0 | Keys: 0

Delete A User –

DELETE USER ACCOUNT

WARNING: This will permanently delete the user and all data.

Enter username to delete: User

Confirm deletion of user 'User'

Type 'y' to confirm: y

[SUCCESS] Success! User 'User' deleted.

Force Password Reset –

FORCE PASSWORD RESET

This will immediately reset the user's password.

Enter username: Faheem

Password Requirements:

- * Minimum 8 characters
- * At least one uppercase letter
- * At least one number

New Password:

[SUCCESS] Password reset for 'Faheem'!

New Password: Faheem1234

View All Files–

ALL ENCRYPTED FILES

- * Faheem_sample.py.enc
- * Superduash_logo.png.enc
- * Superduash_sample (1).txt.enc
- * Superduash_sample.mp4.enc
- * Superduash_sample.txt.enc

Delete Any File –

DELETE ANY FILE

SELECT FILE

[1] sample.py (shared)

```
[2] logo.png (shared)
[3] sample (1).txt (shared)
[4] sample.mp4 (shared)
[5] sample.txt (shared)
```

Select file (1-5): 3

Delete 'Superduash_sample (1).txt.enc'?
Type 'y' to confirm: y

[SUCCESS] Success! File deleted.

View All Logs –

SYSTEM LOGS

```
[2025-11-07 06:15:57] - USER: Admin - ACTION: CLEAR_LOGS - DETAILS: All
logs cleared
[2025-11-07 06:17:38] - USER: Admin - ACTION: LOGIN - DETAILS: User logged
in as ADMIN
[2025-11-07 06:17:44] - USER: Admin - ACTION: CLEAR_BACKUPS - DETAILS:
Cleared 5 backup files
[2025-11-07 06:18:02] - USER: Admin - ACTION: CLEANUP - DETAILS: Removed 2
orphaned files
[2025-11-07 06:18:06] - USER: Admin - ACTION: LOGOUT - DETAILS: Admin
logged out
[2025-11-07 06:18:12] - USER: Faheem - ACTION: LOGIN - DETAILS: User logged
in as USER
```

Export Logs –

EXPORT LOGS

Exporting logs...

[SUCCESS] Success! Logs exported to: C:\Users\Superduash\Downloads\Cloud encryption\cstorage\backups\logs_export_20251107_081729.txt

Clear All Logs –

CLEAR LOGS

WARNING: This action is permanent.

Type 'y' to confirm: y

[SUCCESS] Logs cleared successfully.

Clear All Backups –

CLEAR ALL BACKUPS

WARNING: This will permanently delete all backup files.

This includes:

- Key backups
- Log exports
- Metadata exports

Type 'y' to confirm: y

[SUCCESS] Success! Cleared 3 backup file(s).

Warning! Delete All Data –

DELETE ALL FILES & KEYS

CRITICAL WARNING

This will permanently delete:

- ALL encrypted files
- ALL encryption keys
- ALL file metadata

This action CANNOT be undone!

All users will lose access to their files.

Type 'DELETE ALL' to confirm: DELETE ALL

Are you absolutely sure? (yes/no): yes

[SUCCESS] Success! Deleted 4 file(s) and 6 key(s).

System Statistics –

SYSTEM STATISTICS

Total Users: 4

Total Files: 0

Total Keys: 0

Encrypted Path: C:\Users\Superduash\Downloads\Cloud
encryption\cstorage\encrypted

Decrypted Path: C:\Users\Superduash\Downloads\Cloud
encryption\cstorage\decrypted

User Activity Summary –

USER ACTIVITY SUMMARY

User: Admin
Logins: 0 | Uploads: 0
Downloads: 0 | Deletes: 1

Export Metadata –

EXPORT METADATA

Exporting metadata...

[SUCCESS] Success! Metadata exported to:
C:\Users\Superduash\Downloads\Cloud
encryption\cstorage\backups\metadata_export_20251107_082103.json

System Cleanup Utility –

SYSTEM CLEANUP UTILITY

Scanning for orphaned files...

No orphaned files found. System is clean.

Maintenance Mode –

SYSTEM MAINTENANCE MODE

Current Status: DISABLED

[1] Enable Maintenance Mode
[2] Disable Maintenance Mode
[3] Cancel

Select option: 1

[SUCCESS] Maintenance mode enabled.

UPLOAD FILE

Uploads are currently disabled during maintenance.

Exiting Application –

Thank you for using C-Encrypt!

Goodbye!

8. Conclusion:

This mini project demonstrates how Python can be used effectively for secure cloud-based data management. The developed system ensures data privacy through encryption, simplifies file sharing, and supports user and admin-level control. It shows how security and usability can be combined to solve real-world file management challenges in businesses and organizations.